

# Reentrancy

Reentrancy is probably the most studied vulnerability in the smart contract field. The main idea behind it is that smart contracts can interact with other smart contracts within a function via a `call` statement. It is recommended to have such interactions with other smart contracts *after* making changes to the current state of the calling smart contract. The main reason is that a `call` from a contract  $c$  to a contract  $c'$  may give control to  $c'$ , and thus  $c'$  can call  $c$  again. The classical example here is a simple deposit/withdraw contract  $c$  that allows a malicious smart contract  $c'$  to first deposit some cryptocurrency and then withdraw it. If  $c$  makes state changes *after* the `call` (which performs the actual withdrawal transfer), such as setting the balance of  $c'$  to 0 after the withdrawal, these changes occurring too late can be exploited to drain the funds out of  $c$ .

**Specification primitives** Our defect specifications rely on standard Solidity datatypes— `Contract`, `Method`, `Address`, `Env(ironment)`, and `State`. A brief description of the necessary predicates and functions over these types is included in Table 1. The evaluation of these primitives is stratified: structural properties are derived statically via Slither, while behavioral properties are verified at runtime via Forge/Kontrol assertions within the LLM-generated test harness.

**Table 1.** Primitives for Defect Specification

| Symbol                                                                 | Semantics                                                                                                                                                                                                                                            |
|------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Static Analysis (Slither)</i>                                       |                                                                                                                                                                                                                                                      |
| <code>methods(<math>c</math>)</code>                                   | The set of public/external methods defined in contract $c$ .                                                                                                                                                                                         |
| <code>YieldsControl(<math>c, c', \sigma</math>)</code>                 | true if, in state $\sigma$ , contract $c$ executes an instruction (e.g., <code>CALL</code> , <code>STATICCALL</code> , or <code>DELEGATECALL</code> ) that transfers execution flow to an external contract $c'$ , creating a window for a callback. |
| <i>Runtime Verification (LLM-generated based on specific template)</i> |                                                                                                                                                                                                                                                      |
| <code>Interm(<math>\sigma_i, \sigma, m</math>)</code>                  | Holds if $\sigma_i$ is an intermediate state for the execution of method $m$ starting from state $\sigma$                                                                                                                                            |
| <code>Exec(<math>m, \sigma, \sigma'</math>)</code>                     | Holds if invoking $m$ in state $\sigma$ successfully transitions to $\sigma'$ .                                                                                                                                                                      |
| <code>Spec(<math>m, \sigma</math>)</code>                              | Holds if the functional specification of $m$ is satisfied in state $\sigma$ .                                                                                                                                                                        |
| <code>Profit(<math>\sigma</math>)</code>                               | Holds if the attacker’s balance or internal accounting shares in state $\sigma$ have increased relative to their initial deposit, indicating a successful exploit of state inconsistency.                                                            |

**Domain Conceptual Model.** A contract  $c$  is *reentrancy-vulnerable* if there are (1) a method  $m$  of  $c$ , whose execution is supposed to be atomic, and (2) a state

$\sigma$  in which  $m$  can be invoked, and (3) an intermediate state  $\sigma_i$  during the execution of  $m$ ; such that

(1) an external method  $m'$  can be invoked in state  $\sigma_i$ , and (2) execution of  $m'$  ends in a state  $\sigma'_f$ , and (3) the state  $\sigma'_f$  does not satisfy the contract invariant expected to hold after atomic execution of  $m$ . This allows the attacker to drain funds or manipulate logic by performing additional calls to the vulnerable contract before the state update is finalized.

### Formal Specification.

Reentrancy is defined as the existence of a “Callback Window” where a transient invariant is suspended, allowing attackers to exploit an inconsistent state via any method:

$$\begin{aligned} \exists m, m' \in \text{methods}(c). \exists \mathcal{I} : \text{State} \rightarrow \mathbb{B}. \exists \sigma, \sigma_i \in \text{State}. \text{Reentrant}(c) \equiv \\ \mathcal{I}(\sigma) \wedge \text{Interm}(\sigma_i, \sigma, m) \wedge \neg \mathcal{I}(\sigma_i) \wedge (\exists c' \neq c, \text{YieldsControl}(c, c', \sigma_i)) \wedge \\ (\exists \sigma'_f, \text{Exec}(m', \sigma_i, \sigma'_f) \wedge \text{Profit}(\sigma'_f) \wedge \neg \text{Spec}(c, \sigma'_f)) \end{aligned}$$

where  $\mathcal{I} : \text{State} \rightarrow \mathbb{B}$  represents a global consistency property (e.g., “Total Supply = Sum of Balances”) that is expected to hold at transaction boundaries but may be temporarily suspended during external calls.

The vulnerability is defined as the ability to execute *any* method (including view functions for read-only reentrancy) while the global consistency property is false, provided that has yielded control to an external contract.

### Specific test template design

**Testing Goal** Prove that execution control can be hijacked to perform recursive calls, thereby breaking the intended atomicity of a sensitive method.

**Setup** The LLM generates a malicious attacker contract (e.g., `ExploitContract`) that tracks the number of re-entries using a dedicated state variable, such as `reentrancyCount`.

**Callback Logic** Within the `receive()` or `fallback()` function of the attacker contract, the LLM instantiates logic to re-invoke the victim’s vulnerable method. This callback increments the `reentrancyCount` upon each successful entry to provide a trace-based proof of the exploit.

**Action** The test harness utilizes Forge cheatcodes, specifically `vm.deal` to fund the attacker and `vm.prank` to initiate the transaction from the malicious address, triggering the victim’s external call.

**Assertion** The defect is validated by asserting `reentrancyCount > 1`. This provides a deterministic, trace-based verification that the intermediate state  $\sigma_i$  was reached and that the contract’s control flow was affected.

**Template Derivation and LLM Instantiation** The conceptual test design is materialized into a standardized Foundry test template (e.g., `Reentrancy.t.sol`). This template consists of two primary architectural components: a main test

harness inheriting from Foundry’s `Test` contract, and an `Attacker` contract designed to execute the exploit. To bridge the gap between static analysis and generation, the template embeds explicit `[LLM_INSTRUCTION]` comments alongside structural markers for the Setup, Action, Assertion, and Callback phases.

During the instantiation phase, the LLM acts as a code-completion engine guided by these embedded instructions. The process unfolds as follows:

**Target Initialization (Setup):** The LLM is instructed to import the target smart contract artifact and initialize both the victim and attacker instances. It automatically applies cheatcodes like `vm.deal` to fund the contracts with the necessary Ether to simulate a realistic network state.

**Input Parameterization:** To support both fuzzing and symbolic execution, the LLM parameterizes the test function with variable arguments (e.g., `attackVal`). Crucially, the template guides the LLM to constrain these inputs using `vm.assume` to prevent trivial environment failures, such as integer overflows or balance underflows.

**Exploit Synthesis (Action & Callback):** The LLM synthesizes the contract-specific attack logic within the `Attacker` contract. It populates an `attack()` function to initiate the external call and implements the malicious re-entrant loop inside the `receive()` or `fallback()` method. This callback increments a dedicated `reentrancyCount` state variable.

**Standardized Validation (Assertion):** The framework enforces a uniform validation mechanism. The test harness concludes by using the assert primitive `assertTrue(_attacker.exploitSucceeded())`. The `exploitSucceeded` flag is typically set to true when the `reentrancyCount` exceeds a threshold, providing a deterministic, trace-based proof that the framework successfully generated a valid exploit.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity 0.8.29;
3
4 import {Test} from "../lib/forge-std/src/Test.sol";
5 // [LLM_INSTRUCTION]: Import the artifact of the contract being tested.
6 // [LLM_INSTRUCTION]: If you need to manipulate private state directly,
7 // [LLM_INSTRUCTION]: Import the artifact of the contract being tested. The
8 // [LLM_INSTRUCTION]: Use StdStorage if needed for complex state setup:
9 // [LLM_INSTRUCTION]: Name the contract 'TestReentrancy[ContractName]'
10 // [Testing Goal] Prove that execution control can be hijacked to perform
11 // recursive calls, thereby breaking the intended atomicity of a sensitive
12 // method.
13
14 // [LLM_INSTRUCTION]: Name the contract 'TestReentrancy[ContractName]'
15 contract TestReentrancyTemplate is Test {
16     // [LLM_INSTRUCTION]: Use StdStorage if needed for complex state setup:
17     // using stdStorage for StdStorage;
18
19     // -----
20 }

```

```

19 // [Setup] Declare the contract under test and the malicious attacker
    contract.
20 // -----
21
22 // [LLM_INSTRUCTION]: Declare the contract under test variable (e.g.
    ReentrancySimple public _contractUnderTest)
23
24 Attacker public _attacker;
25
26 // ----- [/Setup]
27
28 function setUp() public {
29 // -----
30 // [Setup] Initialize the victim contract and the attacker contract.
31 // -----
32
33 // [LLM_INSTRUCTION]: Initialize the contract under test.
34 // 1. If the constructor has parameters, use valid concrete values
    here (or setup variables).
35 // 2. If the constructor is payable, use 'vm.deal(address(this),
    amount)' before 'new'.
36
37 // _contractUnderTest = new ReentrancySimple();
38
39 _attacker = new Attacker(address(_contractUnderTest));
40
41 // ----- [/Setup]
42 }
43
44 // [LLM_INSTRUCTION]: Add Fuzz/Symbolic arguments to the test function.
45 // 1. 'attackVal': The value sent/withdrawn during the attack (e.g.
    amount).
46 // 2. 'stateVal': Any value needed to configure the initial state (e.g.
    initial balance).
47 // Example: function test_attackerCallsWithdrawMultipleTimes(uint256
    attackVal, uint256 stateVal) public {
48 function test_attackerCallsMultipleTimes(uint256 attackVal) public {
49
50 // -----
51 // [Setup] Constrain Fuzz/Symbolic values and configure pre-
    conditions.
52 // -----
53
54 // [LLM_INSTRUCTION]: Constrain the Fuzz/Symbolic values using 'vm.
    assume'.
55 // WARNING: Avoid Integer Overflow. If attackVal is added to balances
    , ensure it doesn't wrap around.
56 // WARNING: Avoid Balance Overflow. Do not use type(uint256).max for
    ETH amounts.
57 // WARNING: Avoid Integer Underflow or Overflow during the attack
    scenario.
58 // SUGGESTION: Use vm.assume to limit attackVal to a safe range.
59 // SUGGESTION: Use type(uint128).max as a safe upper bound for
    balance math.
60 // SUGGESTION: Limit the exploit to 1 initial call and 1 re-entrant
    call for simplicity.
61
62 //vm.assume(attackVal > 0.01 ether && attackVal < type(uint128).max);
63
64 // [LLM_INSTRUCTION]: Does the vulnerable function require specific
    state?

```

```

65         // (e.g., specific balance, time passed, authorized user, boolean
66         //      flag).
67
68         // STRATEGY A: Public Methods (Preferred)
69         // Call public setters or 'deposit' functions.
70         // Example: _contractUnderTest.setVal(fuzzArg);
71
72         // STRATEGY B: Storage Manipulation (For Private/Hard-to-reach State)
73         // If the variable is private or requires complex logic to set, use '
74         //      vm.store' or 'stdstore'.
75         // Example:
76         // stdstore.target(address(_contractUnderTest)).sig("myVar()").
77         //      checked_write(fuzzArg);
78
79         // TIMING: If logic depends on time, use vm.warp(block.timestamp +
80         //      100);
81
82         // ----- [/Setup]
83         // -----
84         // [Action] Fund contracts using vm.deal and trigger the attack
85         //      sequence.
86         // -----
87
88         // [LLM_INSTRUCTION]: Fund the contracts using vm.deal.
89         // Use the fuzzed 'attackVal' to ensure sufficient balance.
90
91         // A. Fund the Victim (so it has ETH to steal)
92         vm.deal(address(_contractUnderTest), attackVal * 2);
93
94         // B. Fund the Attacker
95         vm.deal(address(_attacker), attackVal * 2);
96
97         // C. CRITICAL: Fund the Test Contract (address(this))
98         // The test contract needs funds because it sends 'attackVal' to the
99         //      Attacker in the setup step below.
100        // Without this, the test will fail with 'EvmError: OutOfFunds' if
101        //      attackVal > default test balance.
102        vm.deal(address(this), attackVal * 2);
103
104        // [LLM_INSTRUCTION]: If the contract requires specific state (e.g.
105        //      logic dependent on block.timestamp), set it here.
106
107        // [LLM_INSTRUCTION]: Trigger the attack sequence.
108        // Pass the Fuzz/Symbolic values to the setup and attack functions.
109        // Note: The {value: attackVal} here draws from address(this), which
110        //      is why we funded it above.
111        _attacker.setupAttack{value: attackVal}(attackVal);
112        _attacker.attack(attackVal);
113
114        // ----- [/Action]
115        // -----
116        // [Assertion] Validate that the exploit succeeded. The
117        //      exploitSucceeded
118        //      flag is most commonly set when reentrancyCount > 1, but may also
119        //      incorporate additional contract-specific conditions (e.g., balance
120        //      drained beyond expected, state variables corrupted). This provides
121        //      deterministic, trace-based verification that the intermediate
122        //      state
123        //      sigma_i was reached and the contract's control flow was
124        //      manipulated.
125        // -----

```

```

114
115     assertTrue(_attacker.exploitSucceeded(), "Reentrancy attack failed:
116         exploitSucceeded flag was not set.");
117 // ----- [/Assertion]
118 }
119
120     // Keep this so the test contract can receive Ether from the contract
121     // being tested on the "positive" scenario
122     receive() external payable {}
123 }
124 // -----
125 // [Setup] The LLM generates a malicious attacker contract (e.g.,
126 // ExploitContract)
127 // that tracks the number of re-entries using a dedicated state variable
128 // (reentrancyCount).
129 // -----
130 contract Attacker {
131     // [LLM_INSTRUCTION]: Declare the victim contract variable with its
132     // specific type.
133     // ReentrancySimple public _victim;
134
135     // [Setup] Dedicated state variable to track the number of re-entries.
136     uint256 public reentrancyCount;
137     bool public exploitSucceeded;
138
139     // [LLM_INSTRUCTION]: Declare state variables to store the Fuzz/Symbolic
140     // values.
141     // We must store them because 'receive()' cannot accept arguments.
142     // uint256 public storedAttackVal;
143
144     constructor(address victimAddress) {
145         // [LLM_INSTRUCTION]: Cast and store the victim address.
146         // _victim = ReentrancySimple(victimAddress);
147     }
148 // ----- [/Setup]
149 // -----
150     // [Setup] Prepare the attacker's initial state (e.g., deposit into
151     // victim).
152 // -----
153     // [LLM_INSTRUCTION]: Update signature to accept Fuzz/Symbolic values.
154     function setupAttack(uint256 val) public payable {
155         reentrancyCount = 0;
156         exploitSucceeded = false;
157
158         // [LLM_INSTRUCTION]: Perform preparation steps (e.g. deposit) using
159         // the Fuzz/Symbolic value.
160         // Check: If the vulnerability requires a 'deposit' first, do it here
161         // _victim.deposit{value: val}();
162     }
163 // ----- [/Setup]
164 // -----
165

```

```

166 // [Action] Initiate the exploit by calling the victim's vulnerable
167 // function.
168 // -----
169 // [LLM_INSTRUCTION]: Update signature to accept Fuzz/Symbolic values.
170 function attack(uint256 val) public {
171 // [LLM_INSTRUCTION]: 1. Store the value for the re-entrant call if
172 // needed.
173 // storedAttackVal = val;
174 // [LLM_INSTRUCTION]: 2. Call the vulnerable function.
175 // This initiates the external call that will trigger the callback.
176 // _victim.withdraw(); (Or _victim.withdraw(val) if it takes args)
177 //
178 // [Assertion] Set exploitSucceeded based on the success condition.
179 // [LLM_INSTRUCTION]: The most common condition is reentrancyCount >
180 // 1.
181 // However, you may also include additional contract-specific checks
182 // (e.g., balance drained, state variables corrupted).
183 // Example: if (reentrancyCount > 1) { exploitSucceeded = true; }
184 }
185 // ----- [/Action]
186 // -----
187 // [Callback Logic] Within receive()/fallback(), re-invoke the victim's
188 // vulnerable method. Increment reentrancyCount upon each successful
189 // entry
190 // to provide a trace-based proof of the exploit.
191 // -----
192 receive() external payable {
193 reentrancyCount++;
194 if (reentrancyCount < 2) {
195 // [LLM_INSTRUCTION]: Re-enter the victim's vulnerable method
196 // using the stored Fuzz/Symbolic value.
197 // _victim.withdraw(); (Or _victim.withdraw(storedAttackVal))
198 }
199 }
200 // ----- [/Callback Logic]
201 }

```

Listing 1.1. Reentrancy Multi-Shot test template